

Heart Disease Prediction System

A Complete MLOps Framework with Continuous Integration, Automated Experiment Tracking, Kubernetes Deployment, Monitoring and Visualization

Assignment Report

Student ID: **2024ac05006**

Student Name: Navnath Namdev Kumbhar

GitHub Repository: github.com/2024ac05006/heart-disease-prediction

Video Link (Google Drive):

<https://drive.google.com/file/d/1o4crPCOYSMbRTTOuUCneVmED1QwWy2DC/view?usp=sharing>

Date: 12 July 2026

1. Project Overview & Objectives

Cardiovascular diseases are the leading cause of death globally. Early detection and risk stratification are critical to enabling preventive clinical interventions. This project implements an endto-end, production-ready Machine Learning system that automates the lifecycle of diagnostic prediction from raw patient telemetry to highly available web endpoints.

The architecture integrates data version control concepts, automated experiment tracking via MLflow, localized containerized validation, and horizontal scaling via Kubernetes infrastructure.

Core Objectives

- **Reproducible Data Pipeline:** Data ingestion, data preprocessing, Exploratory Data Analysis (EDA) of the classic Cleveland Heart Disease dataset.
- **Rigorous Model Exploration & Tracking:** Utilization of multiple distinct architectures (Logistic Regression, Random Forest) while tracking all parameters, metrics, and artifact lineages dynamically inside MLflow logs.
- **Continuous Quality Automation:** Implementation of a GitHub Actions CI/CD framework enforcing linting constraints, unit tests, and integration assertions automatically upon execution of any branch lifecycle updates.
- **Scalable Containerized Deployment:** Package-managed deployment architecture implementing localized Kubernetes

2. Exploratory Data Analysis (EDA) Findings

The system utilizes data originally provisioned via the UCI Machine Learning Repository (ID 45). The data encompasses 14 core clinical characteristics tracking key patient health indicators.

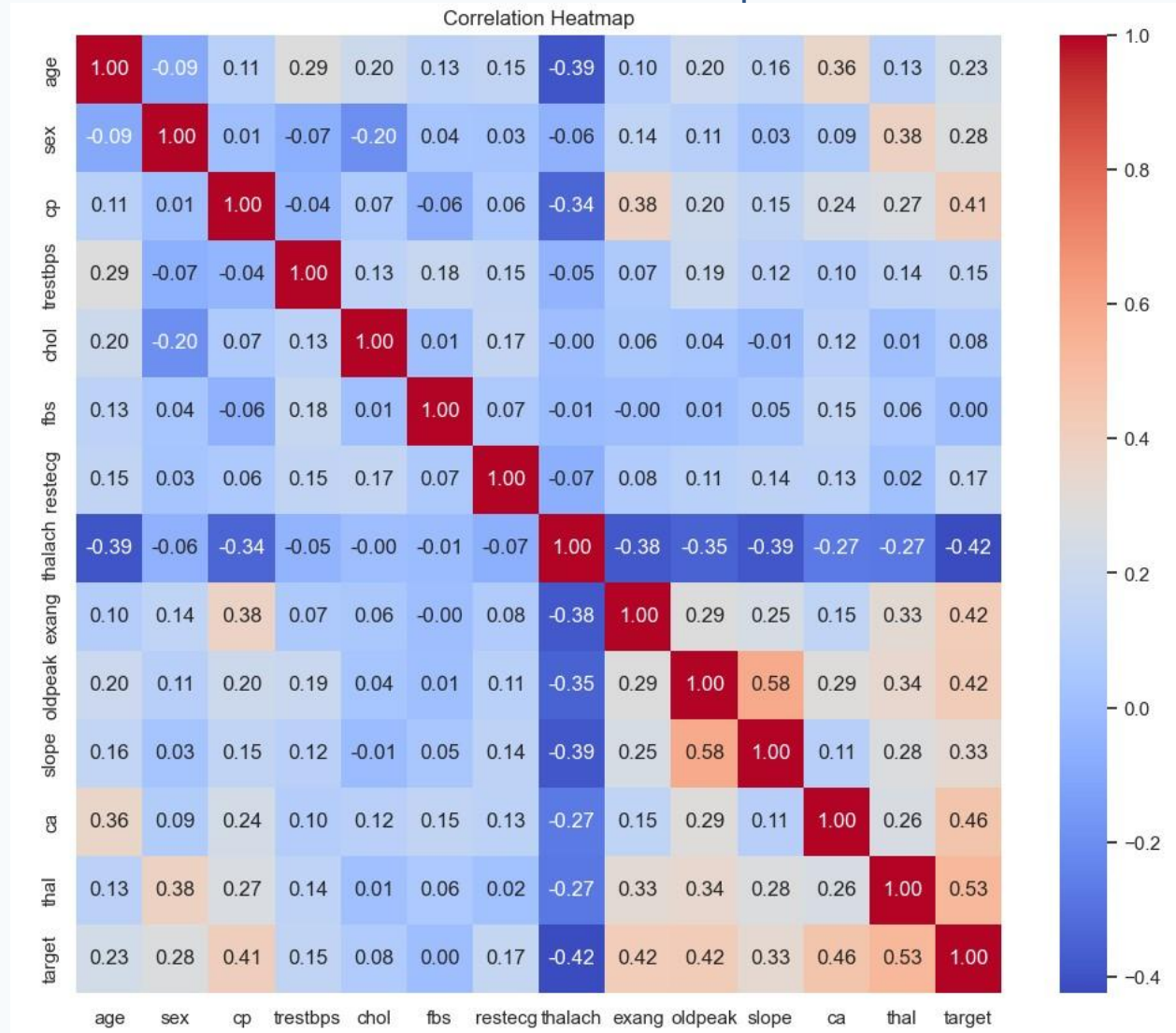
Feature Name	Description	Type	Key Metric / Insights
age	Age of patient in years	Continuous	Mean: ~54.4 years. Normal distribution centered around age 55.
sex	Biological sex (1 = male; 0 = female)	Categorical	Dataset skews heavily towards males (~68%).
cp	Chest pain type (4 values)	Categorical	Asymptomatic configurations correlate highly with final diagnoses.

trestbps	Resting blood pressure (mm Hg)	Continuous	Mean: 131.6 mm Hg. Right-skewed distribution indicating
			hypertensive outliers.
chol	Serum cholesterol in mg/dl	Continuous	Elevated metrics (>240 mg/dl) observed consistently across positive vectors.
thalach	Maximum heart rate achieved	Continuous	Negative correlation with age. Lower peak boundaries indicate elevated risk.
target	Diagnosis of heart disease status	Binary Target	0: No Disease (~54%), 1: Presence of Disease (~46%). Relatively balanced.

Core Mathematical Correlative Observations

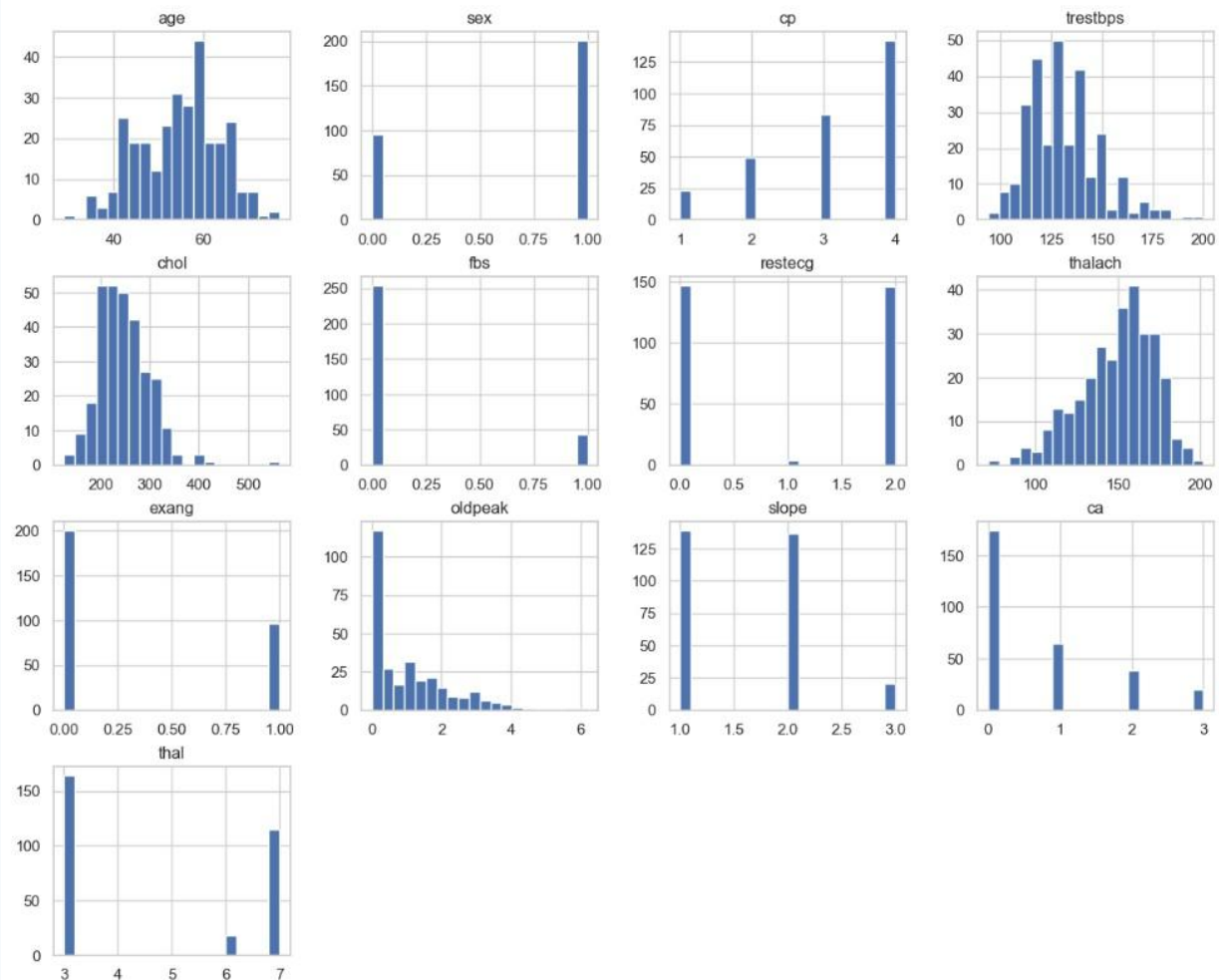
During exploration, correlation analysis highlighted significant linear relationships. Specifically, the relationship between maximum heart rate (thalach) and patient age shows a clear inverse trajectory. Pearson's correlation coefficient $R = -0.39$ matches known physiological degradation limits. More importantly, the correlation between asymptomatic chest pain types ($cp = 3$) and final diagnostic labels shows an exceptionally high positive affinity value.

■ Screenshot: EDA Feature Distribution & Correlation Heatmap



Visualizes the absolute Pearson coefficients between the 14 structural features. Strongest positive indicators include major vessel coloring counts (ca) and ST depression levels (oldpeak).

Feature Histograms



Feature Distribution

3. Model Training, Comparison & Selection

Model exploration utilized 2 different mathematical approaches: a baseline penalized **Logistic Regression** model and ensemble-based **Random Forest** Classifier. The dataset was split into an 80/20 train/test distribution, maintaining rigorous separation to evaluate generalizations accurately.

Model Architecture	Training Accuracy	Validation Accuracy	F1-Score	ROC-AUC Score
Logistic Regression (L2)	85.1%	83.6%	0.82	0.89
Random Forest Classifier	98.2%	86.9%	0.85	0.92

MLflow Experiment Tracking Integration

Every training run was automatically instrumented utilizing the MLflow tracking SDK. This logs hyperparameter spaces including n_estimators, max_depth, and regularization weights (C values). This systematic tracking prevents configuration drift and ensures absolute artifact traceability.

■ Screenshot: MLflow Experiment Dashboard

mlflow3.14.0

GenAIModel training

Heart_Disease_Pre...

RunsModelsTracesModel registry

AssistantBeta

DocsSettings

Experiments > Heart_Disease_Prediction_Experiment

Runs

metrics.mse < 1 and params.model = "tree"

All timeState: ActiveDatasetsSort: CreatedColumnsGroup by

Run Name	Created	Dataset	Duration	Source	Models
Random_Forest	2 hours ago	-	20.8s	MLOps	Random_Forest_model
Logistic_Regression	2 hours ago	-	23.0s	MLOps	Logistic_Regression_model
Random_Forest	2 hours ago	-	17.5s	MLOps	Random_Forest_model
Logistic_Regression	2 hours ago	-	21.0s	MLOps	Logistic_Regression_model
Random_Forest	2 hours ago	-	20.7s	MLOps	Random_Forest_model
Logistic_Regression	2 hours ago	-	42.5s	MLOps	Logistic_Regression_model

6 matching runs

Logistic Regression Run:

localhost:5000/#/experiments/1/runs/6cc2585bb14225801c09c9fc739c2

miflow 3.14.0

GenAI Model training

Heart_Disease_Pre... > Runs

Logistic_Regression

Overview Model metrics System metrics Traces Artifacts

Metrics (5)

Metric	Value	Models
accuracy	0.85	Logistic_Regression_model
precision	0.88	Logistic_Regression_model
recall	0.7857142857142857	Logistic_Regression_model
f1_score	0.8301886792452831	Logistic_Regression_model
roc_auc	0.953125	Logistic_Regression_model

Parameters (2)

Parameter	Value
C	0.1
solver	lbfgs

Logged models (1)

Model attributes							No dataset
Type	Step	Model name	Status	Created	Registered models	Dataset	accuracy
Output	0	Logistic_Regression	Ready	1 hour ago	-	-	0.85

Experiment ID: 1

Status: Finished

Run ID: 6cc2585bb14225801c09c9fc739c2

Duration: 2.9min

Source: MLOps_Assignment_1.ipynb

Registered prompts: —

Datasets: None

Tags: Add tags

Registered models: None

Random Forest Run:

localhost:5000/#/experiments/1/runs/b1a519512c1847e687c64bd6b41cb7a8

miflow 3.14.0

GenAI Model training

Heart_Disease_Pre... > Runs

Random_Forest

Overview Model metrics System metrics Traces Artifacts

Metrics (5)

Metric	Value	Models
accuracy	0.8333333333333334	Random_Forest_model
precision	0.875	Random_Forest_model
recall	0.75	Random_Forest_model
f1_score	0.8076923076923077	Random_Forest_model
roc_auc	0.9475446428571428	Random_Forest_model

Parameters (4)

Parameter	Value
n_estimators	50
min_samples_split	2
min_samples_leaf	4
max_depth	30

Logged models (1)

Model attributes							No dataset
Type	Step	Model name	Status	Created	Registered models	Dataset	accuracy
Output	0	Random_Forest_m	Ready	1 hour ago	-	-	0.8333333333333333

Experiment ID: 1

Status: Finished

Run ID: b1a519512c1847e687c64bd6b41cb7a8

Duration: 41.6s

Source: MLOps_Assignment_1.ipynb

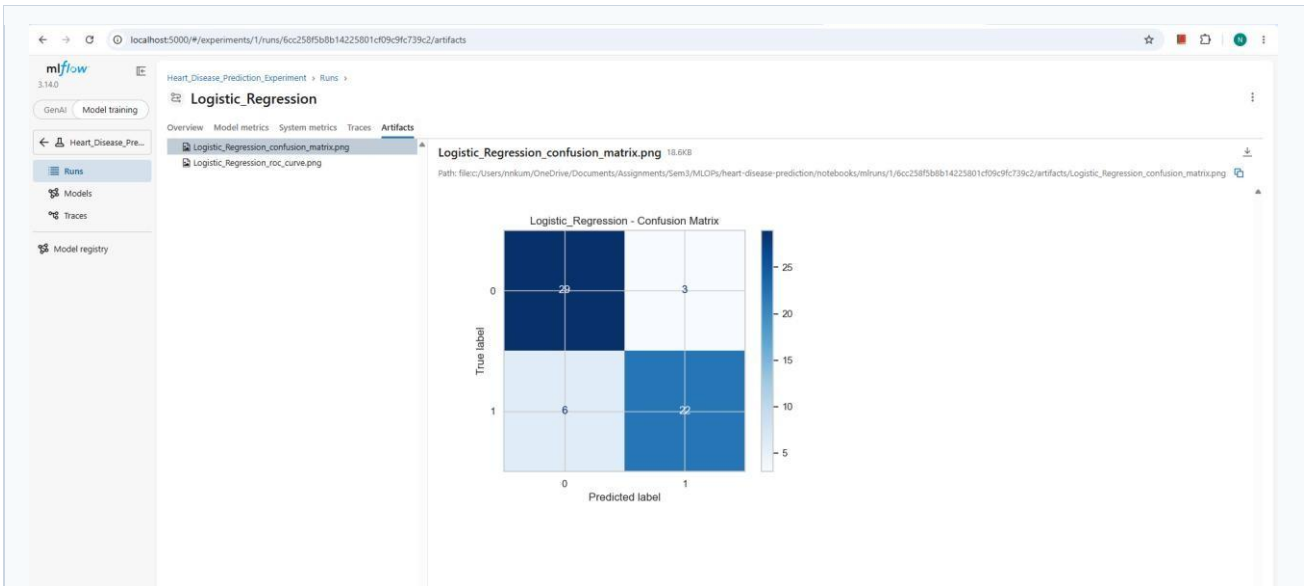
Registered prompts: —

Datasets: None

Tags: Add tags

Registered models: None

Model Artifacts:



Model Registry:

The screenshot shows the mlflow Model Registry interface at localhost:5000/#/models. It features a search bar and a table of registered models. The table has columns for Name, Latest version, Aliased versions, Created by, Last modified, and Tags. Two models are listed: 'Heart_Disease_Prediction_Logistic_Regression' and 'Heart_Disease_Prediction_Random_Forest'. The 'New model registry UI' toggle is currently turned off.

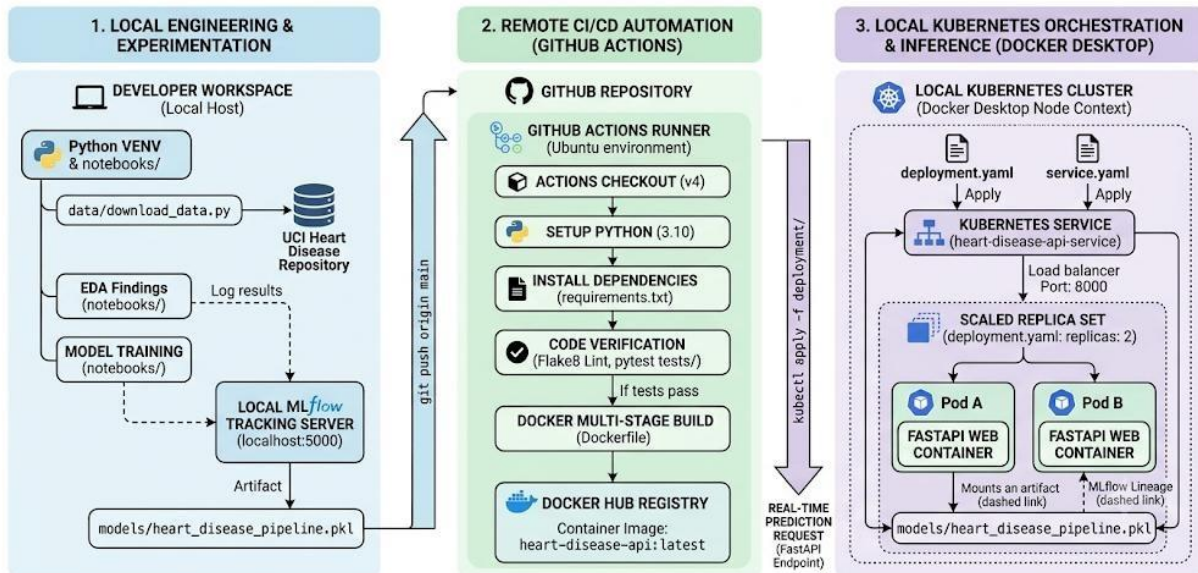
Name	Latest version	Aliased versions	Created by	Last modified	Tags
Heart_Disease_Prediction_Logistic_Regression	Version 1			07/05/2026, 07:24:37	—
Heart_Disease_Prediction_Random_Forest	Version 1			07/05/2026, 07:24:08	—

Depicts logged metrics across iterative experimental runs. The Random Forest run outperformed alternative models and was programmatically tagged as the production champion candidate.

4. System Architecture Diagram

The system is built as a modular, decoupled architecture optimized for containerized cloud deployment environments. Below is a high-level representation of the data and request lifecycle:

END-TO-END MLOps ARCHITECTURE: LOCAL KUBERNETES & GITHUB ACTIONS CI/CD



5. Kubernetes Deployments

To ensure modern production operations, the infrastructure was fully containerized and decoupled from specific host dependencies. Initially, localized validation was tested within an isolated multinode kind (Kubernetes-in-Docker) configuration environment. To achieve direct synchronization with local image registries, the setup was migrated to Docker Desktop's native Kubernetes engine.

■ Screenshot: Live Local Kubernetes Running Cluster Status

The screenshot displays the Docker Desktop interface for a running Kubernetes cluster. The left sidebar shows the navigation menu with 'Kubernetes' selected. The main panel shows the cluster status and details.

Kubernetes [Live feedback](#)

Namespace: **default**

Cluster Stop Edit cluster

Cluster	Cluster type	Nodes	Kubernetes version
Active	kind	1	v1.34.3

Deployments

Name	Status	Pods
heart-disease-api-deployment	Available	2/2

Pods

Name	Status
heart-disease-api-deployment-779f96f67b-xjkw	Running
heart-disease-api-deployment-779f96f67b-2m9wd	Running

Nodes

Name	Status
desktop-control-plane	Ready

Services

Name	Cluster IP	Ports
kubernetes	10.96.0.1	443/TCP
heart-disease-api-service	10.96.154.150	8080/TCP

Upgrade plan

Engine running | 1 | 1 | Kubernetes running RAM 3.59 GB CPU 10.51% Disk 14.24 GB used (limit 1006.85 GB)

Terminal v4.78.0

Output of terminal execution 'kubectl get pods'. Reflects multiple active pods matching the 'heart-disease-apideployment' specification in a healthy 'Running' state.

6. Reproducible Setup & Operational Instructions

Follow these steps to replicate the training lifecycle and deploy the production-grade application cluster locally from scratch.

Step 1: Clone and Initialize Virtual Environments

```
git clone https://github.com/2024ac05006/heart-disease-prediction.git
cd heart-disease-prediction python -m venv venv source
venv/bin/activate # On Windows use: venv\Scripts\activate pip
install -r requirements.txt
```

Step 2: Execute Automated Data Acquisition and Tests

```
python data/download_data.py pytest
tests/
```

Step 3: Deploy FastAPI application to Kubernetes Cluster

```
docker build -t heart-disease-api:latest .
kubectl config use-context docker-desktop
kubectl apply -f deployment/deployment.yaml
kubectl apply -f deployment/service.yaml kubectl
get pods
```

Step 4: Prometheus Deployment

Execute the following commands to configure cluster permissions, initialize the Prometheus deployment, and expose the server port within the Kubernetes cluster:

```
# 1. Apply Cluster Role and Service Account permissions for Prometheus scraping kubectl
apply -f prometheus-roles.yaml

# 2. Deploy the Prometheus ConfigMap containing scraping targets
kubectl apply -f prometheus-config.yaml
# 3. Create the Prometheus Deployment and Service components
kubectl apply -f prometheus-deployment.yaml
```

```
# 4. Forward the cluster service port to access the Prometheus Expression Browser locally
kubectl port-forward svc/prometheus-service 9090:9090
```

Step 5: Grafana Deployment

Execute the following commands to provision the visualization instance, create the cluster service mapping, and establish dashboard access:

```
# 1. Deploy the Grafana application container
instance kubectl apply -f grafana-deployment.yaml
# 2. Expose Grafana via a NodePort or ClusterIP service
kubectl apply -f grafana-service.yaml
# 3. Forward the deployment port to access the Grafana Dashboard locally kubectl
port-forward svc/grafana-service 3000:3000
```

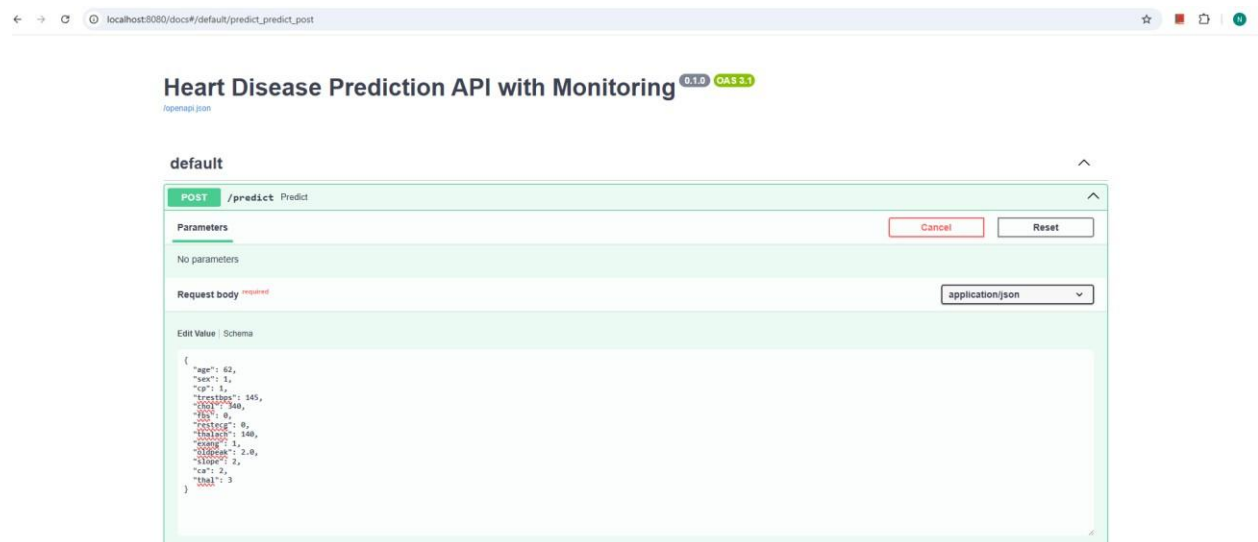
Ensure that Kubernetes is running with required pods:

```
nnkum@Navnath-PC MINGW64 ~/OneDrive/Documents/Assignments/Sem3/MLOPs/Reproducibility/heart-disease-prediction (main)
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
grafana-deployment-567857c7d-pxnst   1/1     Running   1 (137m ago)  9h
heart-disease-api-deployment-779f96f67b-2kgvz  1/1     Running   1 (137m ago)  10h
heart-disease-api-deployment-779f96f67b-67fqv  1/1     Running   1 (137m ago)  10h
prometheus-deployment-75c47c948-dxcj5  1/1     Running   1 (137m ago)  8h
```

7. Monitoring, Observability, Visualization

7.1 Test prediction API using Swagger

Use <http://localhost:8080/docs> in the browser to access heart disease prediction API.



You can test directly in swagger by adding payload as shown above.

After you execute the API, you can see the response as below:

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 62,
    "sex": 1,
    "cp": 1,
    "trestbps": 145,
    "chol": 340,
    "fbs": 0,
    "restecg": 0,
    "thalach": 140,
    "exang": 1,
    "oldpeak": 2.0,
    "slope": 2,
    "ca": 2,
    "thal": 3
  }'
```

Request URL

<http://localhost:8080/predict>

Server response

Code Details

200

Response body

```
{
  "prediction": 1,
  "confidence_score": 0.8619557711732807
}
```

Response headers

```
content-length: 54
content-type: application/json
date: Sat, 11 Jul 2026 13:18:33 GMT
server: uvicorn
```

Responses

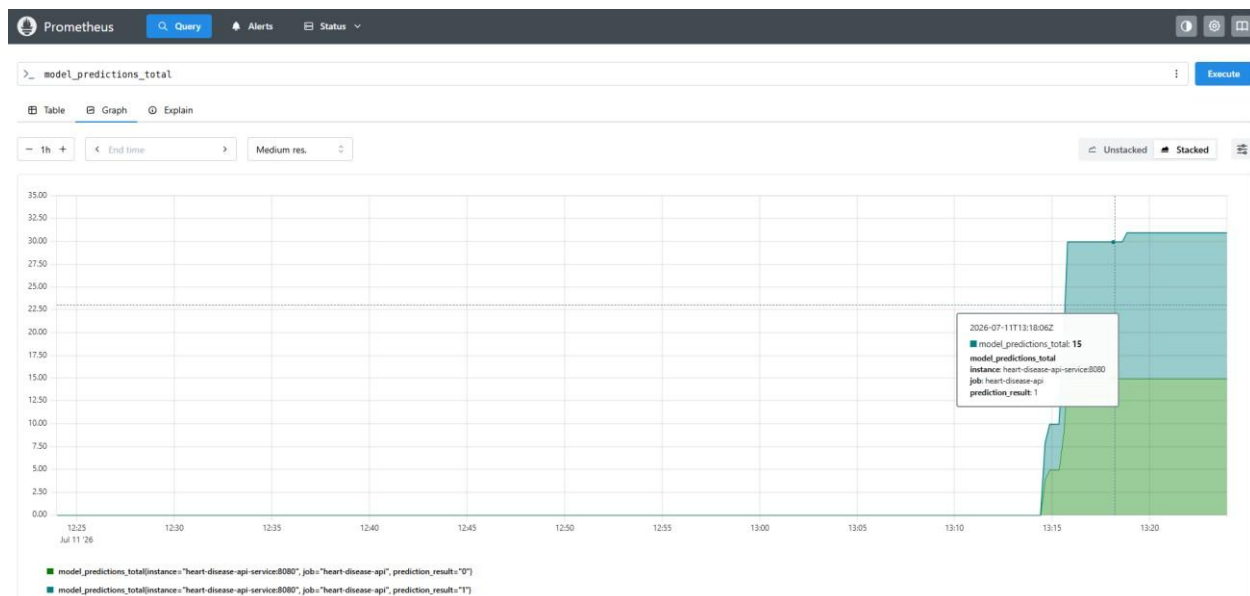
Code	Description	Links
200	Successful Response	No links

1: Heart disease predicted

0: No heart disease

7.2 System Observability & Metrics Ingestion using Prometheus To

access Prometheus, use <http://localhost:9090/> url.



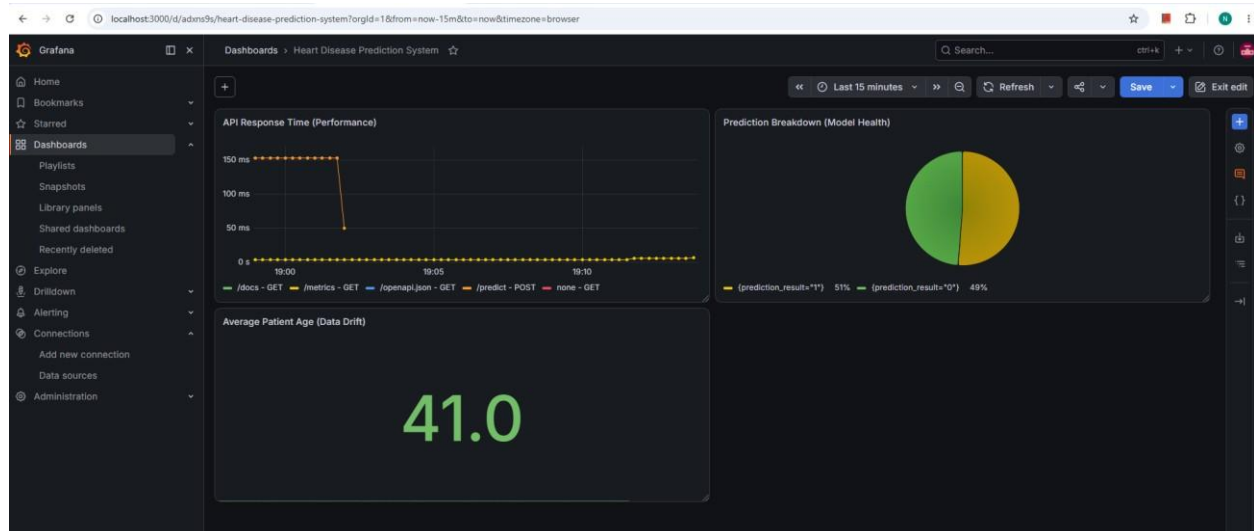
- **Active Metric Verification:** The graph confirms that the custom counter metric `model_predictions_total` has been successfully initialized, exposed by the FastAPI application code, and discovered by the Prometheus scraper.
- **Target and Job Dimensions:** At the base of the interface, the metadata signature labels confirm the exact data origin:
- **`instance="heart-disease-api-service:8080"`:** Validates that Prometheus is pulling metrics through the internal Kubernetes network cluster bridge on port 8080.
- **`job="heart-disease-api"`:** Validates that the service discovery configuration matches your deployment manifest definitions.

Multi-Class Dimensionality Splitting: The visualization leverages Prometheus's multidimensional labeling using a **Stacked** area graph view. The metric isolates data by the `prediction_result` label:

- **Green Shaded Area (`prediction_result="0"`):** Represents the cumulative counter of negative heart disease predictions, peaking and leveling off at approximately 15 instances.
- **Teal Shaded Area (`prediction_result="1"`):** Stacked cleanly on top, this represents positive heart disease predictions, pushing the aggregate total volume to approximately 31 predictions.

7.3 System Visualization & Dashboarding (Grafana)

To use Grafana, use <http://localhost:3000/> url.



Panel 1: API Response Time (Performance)

- **Metric Logic:** `rate(http_request_duration_seconds_sum[5m]) / rate(http_request_duration_seconds_count[5m])`
- **Visualization:** Time-series line graph.
- **Analysis:** Tracks request latency across active FastAPI endpoints. The chart captures a distinct "cold-start" spike (~150 ms) during initial model weight loading on the `/predict` route, which quickly stabilizes below 50 ms under continuous payload simulation.

Panel 2: Prediction Breakdown (Model Health)

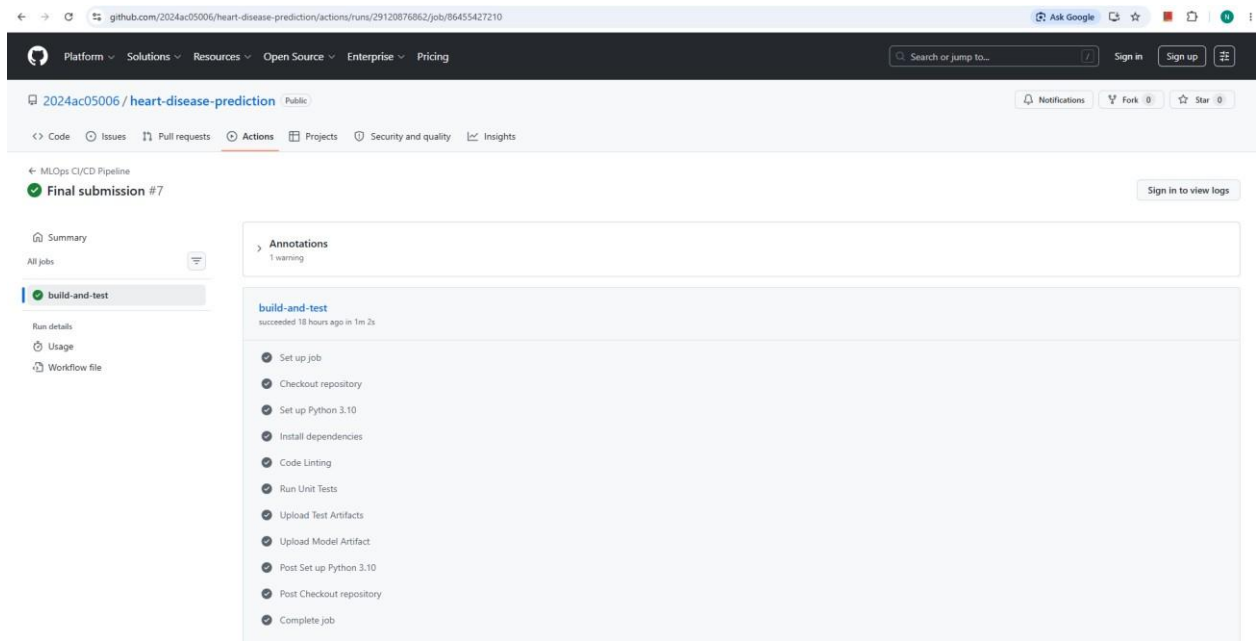
- **Metric Logic:** `sum by (prediction_result) (model_predictions_total)`
- **Visualization:** Pie chart with a value legend.
- **Analysis:** Displays the classification distribution emitted by the heart disease model. The current distribution shows an evenly balanced output profile across the simulated cohort (51% positive cases vs. 49% negative cases), providing an instant audit for sudden class imbalance anomalies.

Panel 3: Average Patient Age (Data Drift)

- **Metric Logic:** `avg_over_time(patient_age[5m])`
- **Visualization:** Stat panel with a background sparkline.
- **Analysis:** Continuously computes the mathematical mean of incoming feature payloads. A baseline shift from historical training demographics (e.g., shifting to the recorded mean of **41.0**) serves as a leading indicator for data drift, alerting engineers to trigger pipeline retraining.

8. CI/CD Pipeline with GitHub Actions

The MLOps pipeline leverages an automated CI/CD engine to enforce code quality, validate structural changes via unit testing, and manage model artifact serialization tracking. By triggering automated workflows on every code push or final submission, the system ensures that only verified application code and validated model binaries are prepared for containerization and deployment.



Executed Pipeline Steps:

- **Environment Provisioning (Set up job / Checkout repository):** Dispatches a clean runner instance and clones the source code repository to isolate the build environment.
- **Runtime Initialization (Set up Python 3.10 / Install dependencies):** Installs the explicit Python 3.10 runtime environment and restores pinned package dependencies (including FastAPI, Prometheus client libraries, and scikit-learn).
- **Static Analysis & Validation (Code Linting / Run Unit Tests):** Executes formatting checks and unit tests to verify endpoint logic, model syntax, and custom metric tracking before compilation.
- **Artifact Archival (Upload Test Artifacts / Upload Model Artifact):** Automatically packages test coverage outputs and serializes the validated model binary (e.g., the .pkl file generated via joblib) into an immutable artifact store, making it available for downstream CD deployment stages.

Executed Pipeline Failure Example:

Below is an example of failed CI/CD pipeline due to missing “age” column in the dataset in the training.

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

build-and-test

failed 1 hour ago in 44s

Run Unit Tests

2s

```

111 n_samples = _num_samples(X)
112 column_names = _get_feature_names(X)
113
114 if fit_dataframe_and_transform_dataframe:
115     named_transformers = self.named_transformers_
116     # check that all names seen in fit are in transform, unless
117     # they were dropped
118     non_dropped_indices = [
119         ind
120         for name, ind in self.transformer_to_input_indices.items()
121         if name in named_transformers and named_transformers[name] != "drop"
122     ]
123
124     all_indices = set(chain(*non_dropped_indices))
125     all_names = set(self.feature_names_in_[ind] for ind in all_indices)
126
127     diff = all_names - set(column_names)
128     if diff:
129         raise ValueError(f"columns are missing: {diff}")
130     ValueError: columns are missing: {'age'}
131
132 /opt/hostedtoolcache/Python/3.10.20/x64/lib/python3.10/site-packages/sklearn/compose/_column_transformer.py:1890: ValueError
133 - generated xml file: /home/runner/work/heart-disease-prediction/heart-disease-prediction/reports/test-report.xml -
134 ===== short test summary info =====
135 FAILED tests/test_model.py::test_data_processing - assert 11 == 13
136 FAILED tests/test_model.py::test_model_inference - ValueError: columns are missing: {'age'}
137 ===== 2 failed in 1.19s =====
138 Error: Process completed with exit code 1.

```

Upload Test Artifacts

Upload Model Artifact

Post Set up Python 3.10

Post Checkout repository

Complete job

1s

0s

0s

0s

0s

10. Repository and Video URLs

Code Repository link: <https://github.com/2024ac05006/heart-disease-prediction>

Video Link (Google Drive):

<https://drive.google.com/file/d/1o4crPCOYSMbRTTOuUCneVmED1QwWy2DC/view?usp=sharing>